

前三条调用语句是通过基类的指针进行的，因为 `fcn` 是虚函数，所以编译器产生的代码将在运行时确定使用虚函数的哪个版本。判断的依据是该指针所绑定对象的真实类型。在 `bp2` 的例子中，实际绑定的对象是 `D1` 类型，而 `D1` 并没有覆盖那个不接受实参的 `fcn`，所以通过 `bp2` 进行的调用将在运行时解析为 `Base` 定义的版本。

621

接下来的三条调用语句是通过不同类型的指针进行的，每个指针分别指向继承体系中的一个类型。因为 `Base` 类中没有 `f2()`，所以第一条语句是非法的，即使当前的指针碰巧指向了一个派生类对象也无济于事。

为了完整地阐明上述问题，我们不妨再观察一些对于非虚函数 `fcn(int)` 的调用语句：

```
Base *p1 = &d2obj; D1 *p2 = &d2obj; D2 *p3 = &d2obj;
p1->fcn(42);           // 错误：Base 中没有接受一个 int 的 fcn
p2->fcn(42);           // 静态绑定，调用 D1::fcn(int)
p3->fcn(42);           // 静态绑定，调用 D2::fcn(int)
```

在上面的每条调用语句中，指针都指向了 `D2` 类型的对象，但是由于我们调用的是非虚函数，所以不会发生动态绑定。实际调用的函数版本由指针的静态类型决定。

覆盖重载的函数

和其他函数一样，成员函数无论是否是虚函数都能被重载。派生类可以覆盖重载函数的 0 个或多个实例。如果派生类希望所有的重载版本对于它来说都是可见的，那么它就需要覆盖所有的版本，或者一个也不覆盖。

有时一个类仅需覆盖重载集合中的一些而非全部函数，此时，如果我们不得不覆盖基类中的每一个版本的话，显然操作将极其烦琐。

一种好的解决方案是为重载的成员提供一条 `using` 声明语句（参见 15.5 节，第 546 页），这样我们就无须覆盖基类中的每一个重载版本了。`using` 声明语句指定一个名字而不指定形参列表，所以一条基类成员函数的 `using` 声明语句就可以把该函数的所有重载实例添加到派生类作用域中。此时，派生类只需要定义其特有的函数就可以了，而无须为继承而来的其他函数重新定义。

类内 `using` 声明的一般规则同样适用于重载函数的名字（参见 15.5 节，第 546 页）；基类函数的每个实例在派生类中都必须是可访问的。对派生类没有重新定义的重载版本的访问实际上是对 `using` 声明点的访问。

15.6 节练习

练习 15.23：假设第 550 页的 `D1` 类需要覆盖它继承而来的 `fcn` 函数，你应该如何对其进行修改？如果你修改之后 `fcn` 匹配了 `Base` 中的定义，则该节的那些调用语句将如何解析？

15.7 构造函数与拷贝控制

622

和其他类一样，位于继承体系中的类也需要控制当其对象执行一系列操作时发生什么样的行为，这些操作包括创建、拷贝、移动、赋值和销毁。如果一个类（基类或派生类）没有定义拷贝控制操作，则编译器将为它合成一个版本。当然，这个合成的版本也可以定义成被删除的函数。



15.7.1 虚析构函数

继承关系对基类拷贝控制最直接的影响是基类通常应该定义一个虚析构函数（参见 15.2.1 节，第 528 页），这样我们就能动态分配继承体系中的对象了。

如前所述，当我们 delete 一个动态分配的对象的指针时将执行析构函数（参见 13.1.3 节，第 445 页）。如果该指针指向继承体系中的某个类型，则有可能出现指针的静态类型与被删除对象的动态类型不符的情况（参见 15.2.2 节，第 530 页）。例如，如果我们 delete 一个 Quote* 类型的指针，则该指针有可能实际指向了一个 Bulk_quote 类型的对象。如果这样的话，编译器就必须清楚它应该执行的是 Bulk_quote 的析构函数。和其他函数一样，我们通过在基类中将析构函数定义成虚函数以确保执行正确的析构函数版本：

```
class Quote {
public:
    // 如果我们删除的是一个指向派生类对象的基类指针，则需要虚析构函数
    virtual ~Quote() = default;    // 动态绑定析构函数
};
```

和其他虚函数一样，析构函数的虚属性也会被继承。因此，无论 Quote 的派生类使用合成的析构函数还是定义自己的析构函数，都将是虚析构函数。只要基类的析构函数是虚函数，就能确保当我们 delete 基类指针时将运行正确的析构函数版本：

```
Quote *itemP = new Quote;           // 静态类型与动态类型一致
delete itemP;                       // 调用 Quote 的析构函数
itemP = new Bulk_quote;             // 静态类型与动态类型不一致
delete itemP;                       // 调用 Bulk_quote 的析构函数
```



如果基类的析构函数不是虚函数，则 delete 一个指向派生类对象的基类指针将产生未定义的行为。

之前我们曾介绍过一条经验准则，即如果一个类需要析构函数，那么它也同样需要拷贝和赋值操作（参见 13.1.4 节，第 447 页）。基类的析构函数并不遵循上述准则，它是一个重要的例外。一个基类总是需要析构函数，而且它能将析构函数设定为虚函数。此时，该析构函数为了成为虚函数而令内容为空，我们显然无法由此推断该基类还需要赋值运算符或拷贝构造函数。

623 虚析构函数将阻止合成移动操作

基类需要一个虚析构函数这一事实还会对基类和派生类的定义产生另外一个间接的影响：如果一个类定义了析构函数，即使它通过=default 的形式使用了合成的版本，编译器也不会为这个类合成移动操作（参见 13.6.2 节，第 475 页）。

15.7.1 节练习

练习 15.24：哪种类需要虚析构函数？虚析构函数必须执行什么样的操作？



15.7.2 合成拷贝控制与继承

基类或派生类的合成拷贝控制成员的行为与其他合成的构造函数、赋值运算符或析构函数类似：它们对类本身的成员依次进行初始化、赋值或销毁的操作。此外，这些合成的成员还负责使用直接基类中对应的操作对一个对象的直接基类部分进行初始化、赋值或销

毁的操作。例如，

- 合成的 Bulk_quote 默认构造函数运行 Disc_quote 的默认构造函数，后者又运行 Quote 的默认构造函数。
- Quote 的默认构造函数将 bookNo 成员默认初始化为空字符串，同时使用类内初始值将 price 初始化为 0。
- Quote 的构造函数完成后，继续执行 Disc_quote 的构造函数，它使用类内初始值初始化 qty 和 discount。
- Disc_quote 的构造函数完成后，继续执行 Bulk_quote 的构造函数，但是它什么具体工作也不做。

类似的，合成的 Bulk_quote 拷贝构造函数使用（合成的）Disc_quote 拷贝构造函数，后者又使用（合成的）Quote 拷贝构造函数。其中，Quote 拷贝构造函数拷贝 bookNo 和 price 成员；Disc_quote 拷贝构造函数拷贝 qty 和 discount 成员。

值得注意的是，无论基类成员是合成的版本（如 Quote 继承体系的例子）还是自定义的版本都没有太大影响。唯一的要求是相应的成员应该可访问（参见 15.5 节，第 542 页）并且不是一个被删除的函数。

在我们的 Quote 继承体系中，所有类都使用合成的析构函数。其中，派生类隐式地使用而基类通过将其虚析构函数定义成=default 而显式地使用。一如既往，合成的析构函数体是空的，其隐式的析构部分负责销毁类的成员（参见 13.1.3 节，第 444 页）。对于派生类的析构函数来说，它除了销毁派生类自己的成员外，还负责销毁派生类的直接基类；该直接基类又销毁它自己的直接基类，以此类推直至继承链的顶端。

C++
11

如前所述，Quote 因为定义了析构函数而不能拥有合成的移动操作，因此当我们移动 Quote 对象时实际使用的是合成的拷贝操作（参见 13.6.2 节，第 477 页）。如我们即将看到的那样，Quote 没有移动操作意味着它的派生类也没有。

624

派生类中删除的拷贝控制与基类的关系

就像其他任何类的情况一样，基类或派生类也能出于同样的原因将其合成的默认构造函数或者任何一个拷贝控制成员定义成被删除的函数（参见 13.1.6 节，第 450 页和 13.6.2 节，第 475 页）。此外，某些定义基类的方式也可能导致有的派生类成员成为被删除的函数：

C++
11

- 如果基类中的默认构造函数、拷贝构造函数、拷贝赋值运算符或析构函数是被删除的函数或者不可访问（参见 15.5 节，第 543 页），则派生类中对应的成员将是被删除的，原因是编译器不能使用基类成员来执行派生类对象基类部分的构造、赋值或销毁操作。
- 如果在基类中有一个不可访问或删除掉的析构函数，则派生类中合成的默认和拷贝构造函数将是被删除的，因为编译器无法销毁派生类对象的基类部分。
- 和过去一样，编译器将不会合成一个删除掉的移动操作。当我们使用=default 请求一个移动操作时，如果基类中的对应操作是删除的或不可访问的，那么派生类中该函数将是被删除的，原因是派生类对象的基类部分不可移动。同样，如果基类的析构函数是删除的或不可访问的，则派生类的移动构造函数也将是被删除的。

举个例子，对于下面的基类 B 来说：

```
class B {
public:
    B();
```

```

        B(const B&) = delete;
        // 其他成员, 不含有移动构造函数
    };
    class D : public B {
        // 没有声明任何构造函数
    };
    D d; // 正确: D 的合成默认构造函数使用 B 的默认构造函数
    D d2(d); // 错误: D 的合成拷贝构造函数是被删除的
    D d3(std::move(d)); // 错误: 隐式地使用 D 的被删除的拷贝构造函数

```

基类 B 含有一个可访问的默认构造函数和一个显式删除的拷贝构造函数。因为我们定义了拷贝构造函数, 所以编译器将不会为 B 合成一个移动构造函数(参见 13.6.2 节, 第 475 页)。因此, 我们既不能移动也不能拷贝 B 的对象。如果 B 的派生类希望它自己的对象能被移动和拷贝, 则派生类需要自定义相应版本的构造函数。当然, 在这一过程中派生类还必须考虑如何移动或拷贝其基类部分的成员。在实际编程过程中, 如果在基类中没有默认、拷贝或移动构造函数, 则一般情况下派生类也不会定义相应的操作。

625 移动操作与继承

如前所述, 大多数基类都会定义一个虚析构函数。因此在默认情况下, 基类通常不含有合成的移动操作, 而且在它的派生类中也没有合成的移动操作。

因为基类缺少移动操作会阻止派生类拥有自己的合成移动操作, 所以当我们确实需要执行移动操作时应该首先在基类中进行定义。我们的 Quote 可以使用合成的版本, 不过前提是 Quote 必须显式地定义这些成员。一旦 Quote 定义了自己的移动操作, 那么它必须同时显式地定义拷贝操作(参见 13.6.2 节, 第 476 页):

```

class Quote {
public:
    Quote() = default; // 对成员依次进行默认初始化
    Quote(const Quote&) = default; // 对成员依次拷贝
    Quote(Quote&&) = default; // 对成员依次拷贝
    Quote& operator=(const Quote&) = default; // 拷贝赋值
    Quote& operator=(Quote&&) = default; // 移动赋值
    virtual ~Quote() = default;
    // 其他成员与之前的版本一致
};

```

通过上面的定义, 我们就能对 Quote 的对象逐成员地分别进行拷贝、移动、赋值和销毁操作了。而且除非 Quote 的派生类中含有排斥移动的成员, 否则它将自动获得合成的移动操作。

15.7.2 节练习

练习 15.25: 我们为什么为 Disc_quote 定义一个默认构造函数? 如果去除掉该构造函数的话会对 Bulk_quote 的行为产生什么影响?



15.7.3 派生类的拷贝控制成员

如我们在 15.2.2 节(第 531 页)介绍过的, 派生类构造函数在其初始化阶段中不但要初始化派生类自己的成员, 还负责初始化派生类对象的基类部分。因此, 派生类的拷贝和

移动构造函数在拷贝和移动自有成员的同时，也要拷贝和移动基类部分的成员。类似的，派生类赋值运算符也必须为其基类部分的成员赋值。

和构造函数及赋值运算符不同的是，析构函数只负责销毁派生类自己分配的资源。如前所述，对象的成员是被隐式销毁的（参见 13.1.3 节，第 445 页）；类似的，派生类对象的基类部分也是自动销毁的。



当派生类定义了拷贝或移动操作时，该操作负责拷贝或移动包括基类部分成员在内的整个对象。

626

定义派生类的拷贝或移动构造函数



当为派生类定义拷贝或移动构造函数时（参见 13.1.1 节，第 440 页和 13.6.2 节，第 473 页），我们通常使用对应的基类构造函数初始化对象的基类部分：

```
class Base { /* ... */ };
class D: public Base {
public:
    // 默认情况下，基类的默认构造函数初始化对象的基类部分
    // 要想使用拷贝或移动构造函数，我们必须在构造函数初始值列表中
    // 显式地调用该构造函数
    D(const D& d): Base(d) // 拷贝基类成员
        /* D 的成员的初始值 */ { /* ... */ }
    D(D&& d): Base(std::move(d)) // 移动基类成员
        /* D 的成员的初始值 */ { /* ... */ }
};
```

初始值 `Base(d)` 将一个 `D` 对象传递给基类构造函数。尽管从道理上来说，`Base` 可以包含一个参数类型为 `D` 的构造函数，但是在实际编程过程中通常不会这么做。相反，`Base(d)` 一般会匹配 `Base` 的拷贝构造函数。`D` 类型的对象 `d` 将被绑定到该构造函数的 `Base&` 形参上。`Base` 的拷贝构造函数负责将 `d` 的基类部分拷贝给要创建的对象。假如我们没有提供基类的初始值的话：

```
// D 的这个拷贝构造函数很可能是不正确的定义
// 基类部分被默认初始化，而非拷贝
D(const D& d) /* 成员初始值，但是没有提供基类初始值 */
{ /* ... */ }
```

在上面的例子中，`Base` 的默认构造函数将被用来初始化 `D` 对象的基类部分。假定 `D` 的构造函数从 `d` 中拷贝了派生类成员，则这个新构建的对象的配置将非常奇怪：它的 `Base` 成员被赋予了默认值，而 `D` 成员的值则是从其他对象拷贝得来的。



在默认情况下，基类默认构造函数初始化派生类对象的基类部分。如果我们想拷贝（或移动）基类部分，则必须在派生类的构造函数初始值列表中显式地使用基类的拷贝（或移动）构造函数。

派生类赋值运算符

与拷贝和移动构造函数一样，派生类的赋值运算符（参见 13.1.2 节，第 443 页和 13.6.2 节，第 474 页）也必须显式地为其基类部分赋值：

```
// Base::operator=(const Base&) 不会被自动调用
```

627

```

D &D::operator=(const D &rhs)
{
    Base::operator=(rhs); // 为基类部分赋值
    // 按照过去的方式为派生类的成员赋值
    // 酌情处理自赋值及释放已有资源等情况
    return *this;
}

```

上面的运算符首先显式地调用基类赋值运算符，令其为派生类对象的基类部分赋值。基类的运算符（应该可以）正确地处理自赋值的情况，如果赋值命令是正确的，则基类运算符将释放掉其左侧运算对象的基类部分的旧值，然后利用 `rhs` 为其赋一个新值。随后，我们继续进行其他为派生类成员赋值的工作。

值得注意的是，无论基类的构造函数或赋值运算符是自定义的版本还是合成的版本，派生类的对应操作都能使用它们。例如，对于 `Base::operator=` 的调用语句将执行 `Base` 的拷贝赋值运算符，至于该运算符是由 `Base` 显式定义的还是由编译器合成的无关紧要。

派生类析构函数

如前所述，在析构函数体执行完成后，对象的成员会被隐式销毁（参见 13.1.3 节，第 445 页）。类似的，对象的基类部分也是隐式销毁的。因此，和构造函数及赋值运算符不同的是，派生类析构函数只负责销毁由派生类自己分配的资源：

```

class D: public Base {
public:
    // Base::~~Base 被自动调用执行
    ~D() { /* 该处由用户定义清除派生类成员的操作 */ }
};

```

对象销毁的顺序正好与其创建的顺序相反：派生类析构函数首先执行，然后是基类的析构函数，以此类推，沿着继承体系的反方向直至最后。

在构造函数和析构函数中调用虚函数

如我们所知，派生类对象的基类部分将首先被构建。当执行基类的构造函数时，该对象的派生类部分是未被初始化的状态。类似的，销毁派生类对象的次序正好相反，因此当执行基类的析构函数时，派生类部分已经被销毁掉了。由此可知，当我们执行上述基类成员的时候，该对象处于未完成的状态。

为了能够正确地处理这种未完成状态，编译器认为对象的类型在构造或析构的过程中仿佛发生了改变一样。也就是说，当我们构建一个对象时，需要把对象的类和构造函数的类看作是同一个；对虚函数的调用绑定正好符合这种把对象的类和构造函数的类看成同一个的要求；对于析构函数也是同样的道理。上述的绑定不但对直接调用虚函数有效，对间接调用也是有效的，这里的间接调用是指通过构造函数（或析构函数）调用另一个函数。

为了理解上述行为，不妨考虑当基类构造函数调用虚函数的派生类版本时会发生什么情况。这个虚函数可能会访问派生类的成员，毕竟，如果它不需要访问派生类成员的话，则派生类直接使用基类的虚函数版本就可以了。然而，当执行基类构造函数时，它要用到的派生类成员尚未初始化，如果我们允许这样的访问，则程序很可能会崩溃。



如果构造函数或析构函数调用了某个虚函数，则我们应该执行与构造函数或析构函数所属类型相对应的虚函数版本。

15.7.3 节练习

练习 15.26: 定义 `Quote` 和 `Bulk_quote` 的拷贝控制成员，令其与合成的版本行为一致。为这些成员以及其他构造函数添加打印状态的语句，使得我们能够知道正在运行哪个程序。使用这些类编写程序，预测程序将创建和销毁哪些对象。重复实验，不断比较你的预测和实际输出结果是否相同，直到预测完全准确再结束。

15.7.4 继承的构造函数

在 C++11 新标准中，派生类能够重用其直接基类定义的构造函数。尽管如我们所知，这些构造函数并非以常规的方式继承而来，但是为了方便，我们不妨姑且称其为“继承”的。一个类只初始化它的直接基类，出于同样的原因，一个类也只继承其直接基类的构造函数。类不能继承默认、拷贝和移动构造函数。如果派生类没有直接定义这些构造函数，则编译器将为派生类合成它们。

C++
11

派生类继承基类构造函数的方式是提供一条注明了（直接）基类名的 `using` 声明语句。举个例子，我们可以重新定义 `Bulk_quote` 类（参见 15.4 节，第 541 页），令其继承 `Disc_quote` 类的构造函数：

```
class Bulk_quote : public Disc_quote {
public:
    using Disc_quote::Disc_quote; // 继承 Disc_quote 的构造函数
    double net_price(std::size_t) const;
};
```

通常情况下，`using` 声明语句只是令某个名字在当前作用域内可见。而当作用于构造函数时，`using` 声明语句将令编译器产生代码。对于基类的每个构造函数，编译器都生成一个与之对应的派生类构造函数。换句话说，对于基类的每个构造函数，编译器都在派生类中生成一个形参列表完全相同的构造函数。

这些编译器生成的构造函数形如：

629

```
derived(parms) : base(args) { }
```

其中，*derived* 是派生类的名字，*base* 是基类的名字，*parms* 是构造函数的形参列表，*args* 将派生类构造函数的形参传递给基类的构造函数。在我们的 `Bulk_quote` 类中，继承的构造函数等价于：

```
Bulk_quote(const std::string& book, double price,
            std::size_t qty, double disc):
    Disc_quote(book, price, qty, disc) { }
```

如果派生类含有自己的数据成员，则这些成员将被默认初始化（参见 7.1.4 节，第 238 页）。

继承的构造函数的特点

和普通成员的 `using` 声明不一样，一个构造函数的 `using` 声明不会改变该构造函数的访问级别。例如，不管 `using` 声明出现在哪儿，基类的私有构造函数在派生类中还是一个私有构造函数；受保护的构造函数和公有构造函数也是同样的规则。

而且，一个 `using` 声明语句不能指定 `explicit` 或 `constexpr`。如果基类的构造函数是 `explicit`（参见 7.5.4 节，第 265 页）或者 `constexpr`（参见 7.5.6 节，第 267

页), 则继承的构造函数也拥有相同的属性。

当一个基类构造函数含有默认实参(参见 6.5.1 节, 第 211 页)时, 这些实参并不会被继承。相反, 派生类将获得多个继承的构造函数, 其中每个构造函数分别省略掉一个含有默认实参的形参。例如, 如果基类有一个接受两个形参的构造函数, 其中第二个形参含有默认实参, 则派生类将获得两个构造函数: 一个构造函数接受两个形参(没有默认实参), 另一个构造函数只接受一个形参, 它对应于基类中最左侧的没有默认值的那个形参。

如果基类含有几个构造函数, 则除了两个例外情况, 大多数时候派生类会继承所有这些构造函数。第一个例外是派生类可以继承一部分构造函数, 而为其他构造函数定义自己的版本。如果派生类定义的构造函数与基类的构造函数具有相同的参数列表, 则该构造函数将不会被继承。定义在派生类中的构造函数将替换继承而来的构造函数。

第二个例外是默认、拷贝和移动构造函数不会被继承。这些构造函数按照正常规则被合成。继承的构造函数不会被作为用户定义的构造函数来使用, 因此, 如果一个类只含有继承的构造函数, 则它也将拥有一个合成的默认构造函数。

15.7.4 节练习

练习 15.27: 重新定义你的 Bulk_quote 类, 令其继承构造函数。



15.8 容器与继承

630

当我们使用容器存放继承体系中的对象时, 通常必须采取间接存储的方式。因为不允许在容器中保存不同类型的元素, 所以我们不能把具有继承关系的多种类型的对象直接存放在容器当中。

举个例子, 假定我们想定义一个 vector, 令其保存用户准备购买的几种书籍。显然我们不应该用 vector 保存 Bulk_quote 对象。因为我们不能将 Quote 对象转换成 Bulk_quote (参见 15.2.3 节, 第 534 页), 所以我们将无法把 Quote 对象放置在该 vector 中。

其实, 我们也不应该使用 vector 保存 Quote 对象。此时, 虽然我们可以把 Bulk_quote 对象放置在容器中, 但是这些对象再也不是 Bulk_quote 对象了:

```
vector<Quote> basket;
basket.push_back(Quote("0-201-82470-1", 50));
// 正确: 但是只能把对象的 Quote 部分拷贝给 basket
basket.push_back(Bulk_quote("0-201-54848-8", 50, 10, .25));
// 调用 Quote 定义的版本, 打印 750, 即 15 * $50
cout << basket.back().net_price(15) << endl;
```

basket 的元素是 Quote 对象, 因此当我们向该 vector 中添加一个 Bulk_quote 对象时, 它的派生类部分将被忽略掉(参见 15.2.3 节, 第 535 页)。



当派生类对象被赋值给基类对象时, 其中的派生类部分将被“切掉”, 因此容器和存在继承关系的类型无法兼容。